Experiment No 8

Implementing chunking using NLTK , SpaCy , Stanza in Python

NLTK

```
import nltk
import matplotlib.pyplot as plt

# Download required NLTK packages
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

# --------------------------------
# INPUT
# --------------------------------

sentence = "A beautiful girl is reading a book."

# Tokenization
words = nltk.word_tokenize(sentence)

# POS Tagging
tagged_words = nltk.pos_tag(words)

print("POS Tags:")
print(tagged_words)

# --------------------------------
# CHUNKING
# --------------------------------

grammar = r"""
   NP: {<DT><NN.*>}
   NP: {<JJ><NN.*>}
"""

chunk_parser = nltk.RegexpParser(grammar)

tree = chunk_parser.parse(tagged_words)

print("\nChunk Tree:")
print(tree)

# --------------------------------
# GRAPHICAL TREE
# --------------------------------

def get_width(node):

   # POS tuple such as ('The', 'DT')
   if isinstance(node, tuple):
      return 1

   # Normal word
```

```
    if isinstance(node, str):
        return 1

    # NLTK Tree
    return sum(get_width(child) for child in node)


def draw_node(ax, node, x, y, parent_x=None, parent_y=None):

    # ------------------------------
    # Leaf: POS tuple
    # ------------------------------
    if isinstance(node, tuple):

        word, tag = node

        label = f"{word}\n{tag}"

        # Connection from parent
        if parent_x is not None:
            ax.plot(
                [parent_x, x],
                [parent_y - 0.08, y + 0.08],
                linewidth=1
            )

        ax.text(
            x,
            y,
            label,
            ha="center",
            va="center",
            fontsize=11
        )

        return

    # ------------------------------
    # Leaf: string
    # ------------------------------
    if isinstance(node, str):

        if parent_x is not None:
            ax.plot(
                [parent_x, x],
                [parent_y - 0.08, y + 0.08],
                linewidth=1
            )

        ax.text(
            x,
            y,
            node,
            ha="center",
            va="center",
```

```
            fontsize=11
        )

        return

    # ----------------------------
    # NLTK Tree Node
    # ----------------------------

    if parent_x is not None:
        ax.plot(
            [parent_x, x],
            [parent_y - 0.08, y + 0.08],
            linewidth=1
        )

    # Draw node label
    ax.text(
        x,
        y,
        node.label(),
        ha="center",
        va="center",
        fontsize=13,
        fontweight="bold"
    )

    children = list(node)

    widths = [get_width(child) for child in children]

    total_width = sum(widths)

    current_x = x - total_width / 2

    for child, width in zip(children, widths):

        child_x = current_x + width / 2

        draw_node(
            ax,
            child,
            child_x,
            y - 1,
            x,
            y
        )

        current_x += width


# --------------------------------
# CREATE GRAPH
# --------------------------------
```

```
fig, ax = plt.subplots(figsize=(14, 6))

ax.axis("off")

tree_width = get_width(tree)

draw_node(
    ax,
    tree,
    0,
    0
)

ax.set_xlim(
    -tree_width / 2 - 1,
    tree_width / 2 + 1
)

ax.set_ylim(
    -3.5,
    0.5
)

plt.title(
    "NLTK Chunking Tree",
    fontsize=16,
    fontweight="bold"
)

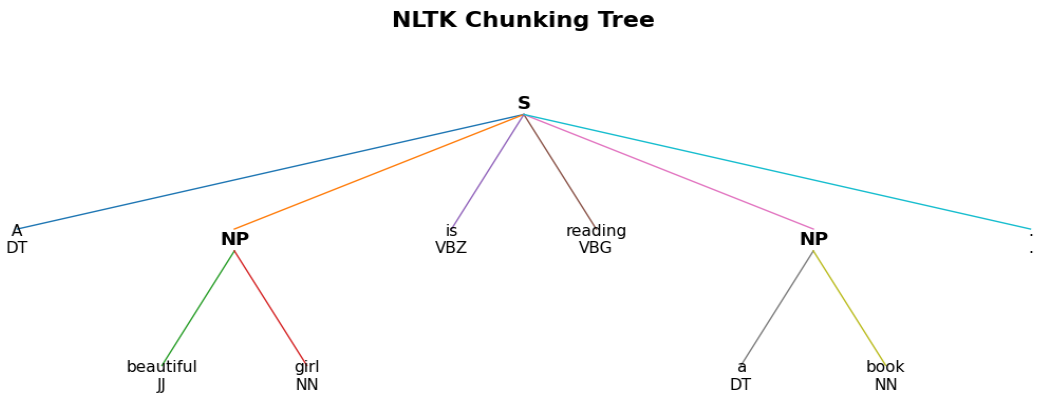
plt.tight_layout()

plt.show()
```

Output

POS Tags:

[('A', 'DT'), ('beautiful', 'JJ'), ('girl', 'NN'), ('is', 'VBZ'), ('reading', 'VBG'), ('a', 'DT'), ('book', 'NN'), ('.', '.')]



SpaCy

```
import spacy
from spacy import displacy

# Load English model
nlp = spacy.load("en_core_web_sm")

# Input sentence
sentence = "A beautiful girl is reading a book."

# Process sentence
doc = nlp(sentence)

# Print POS tags
print("POS Tags:")
for token in doc:
    print(token.text, token.pos_, token.tag_)

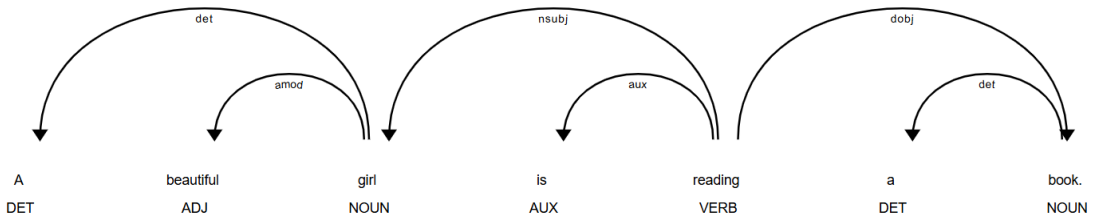
# Print noun chunks
print("\nNoun Chunks:")
for chunk in doc.noun_chunks:
    print(chunk.text)

# Display graphical dependency tree
displacy.render(doc, style="dep", jupyter=True)
```

Output
POS Tags:
A DET DT
beautiful ADJ JJ
girl NOUN NN
is AUX VBZ
reading VERB VBG
a DET DT
book NOUN NN
. PUNCT .

Noun Chunks:
A beautiful girl

a book
A DET beautiful ADJ girl NOUN is AUX reading VERB a DET book. NOUN det amod nsubj aux det dobj



Stanza

```
import stanza
from graphviz import Digraph

# Create Stanza NLP pipeline
nlp = stanza.Pipeline(
    lang="en",
    processors="tokenize,pos,lemma,depparse"
)

# Input sentence
sentence = "A beautiful girl is reading a book."

# Process sentence
doc = nlp(sentence)

# ----------------------------
# POS TAGGING
# ----------------------------

print("POS Tags:")

for sent in doc.sentences:
    for word in sent.words:
        print(
            word.text,
            word.upos,
            word.xpos
        )

# ----------------------------
# DEPENDENCY RELATIONS
# ----------------------------

print("\nDependency Relations:")

for sent in doc.sentences:
    for word in sent.words:
        print(
```

```
            word.id,
            word.text,
            "Head:",
            word.head,
            "Relation:",
            word.deprel
        )

# ----------------------------
# CREATE GRAPH
# ----------------------------

dot = Digraph(
    name="Stanza Dependency Tree",
    format="png"
)

dot.attr(
    rankdir="TB",
    size="12"
)

# Add nodes
for sent in doc.sentences:

    for word in sent.words:

        label = f"{word.text}\n{word.upos}"

        dot.node(
            str(word.id),
            label
        )

# Add dependency edges
for sent in doc.sentences:

    for word in sent.words:

        if word.head != 0:

            dot.edge(
                str(word.head),
                str(word.id),
                label=word.deprel
            )

# Display graph
Dot
```

Output

POS Tags:

A DET DT

beautiful ADJ JJ

girl NOUN NN

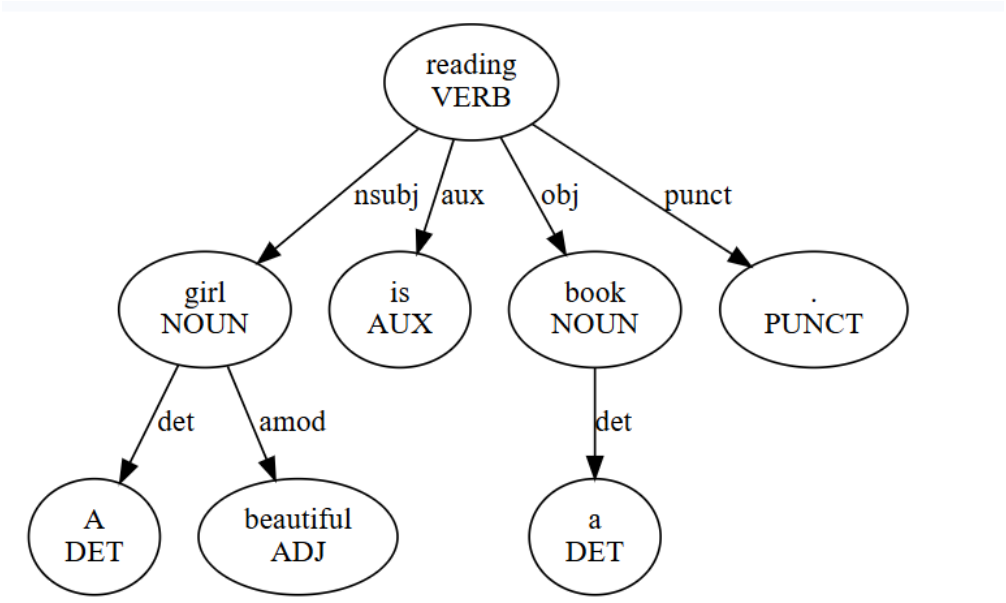
is AUX VBZ

reading VERB VBG

a DET DT

book NOUN NN

. PUNCT .



Hindi
NLTK

```
import nltk
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
from nltk import Tree

# ============================================================
# HINDI POS TAGGING
# ============================================================

sentence = "एक सुंदर लड़की किताब पढ़ रही है।"

# Hindi POS tags
# NLTK does not provide a standard Hindi POS tagger here,
# so tags are provided manually for demonstrating chunking.

words = [
   ("एक", "DT"),
   ("सुंदर", "JJ"),
   ("लड़की", "NN"),
   ("किताब", "NN"),
   ("पढ़", "VB"),
   ("रही", "VBG"),
   ("है", "VBZ"),
   ("।", ".")
]

print("Hindi POS Tags:")
for word, tag in words:
   print(f"{word} -> {tag}")

# ============================================================
# CHUNKING GRAMMAR
# ============================================================

grammar = r"""
   NP: {<DT><JJ>*<NN>}
   NP: {<NN>}
   VP: {<VB><VBG><VBZ>}
"""

# Create chunk parser
chunk_parser = nltk.RegexpParser(grammar)

# Create chunk tree
tree = chunk_parser.parse(words)

print("\nChunk Tree:")
print(tree)

# ============================================================
# HINDI FONT
```

```
# ============================================================

# Find installed Hindi font
font_path = "C:/Windows/Fonts/Nirmala.ttc"

# If Nirmala UI is available, use it
try:
    hindi_font = fm.FontProperties(fname=font_path)
except:
    hindi_font = fm.FontProperties(family="Nirmala UI")

english_font = fm.FontProperties(family="DejaVu Sans")

# ============================================================
# DRAW TREE
# ============================================================

def draw_tree(tree, ax, x=0.5, y=1.0, width=1.0):

    # If node is a Tree
    if isinstance(tree, Tree):

        # NP / VP / S
        ax.text(
            x,
            y,
            tree.label(),
            ha="center",
            va="center",
            fontsize=13,
            fontproperties=english_font,
            bbox=dict(
                boxstyle="round,pad=0.3",
                facecolor="white"
            )
        )

        number_of_children = len(tree)

        if number_of_children > 0:

            child_width = width / number_of_children

            for i, child in enumerate(tree):

                child_x = (
                    x
                    - width / 2
                    + child_width * (i + 0.5)
                )

                child_y = y - 0.18

                # Connecting line
                ax.plot(
```

```
                [x, child_x],
                [y - 0.03, child_y + 0.03],
                "k-"
            )

            draw_tree(
                child,
                ax,
                child_x,
                child_y,
                child_width
            )

    # Leaf node
    else:

        if isinstance(tree, tuple):

            word = tree[0]
            tag = tree[1]

            # Hindi word
            ax.text(
                x - 0.025,
                y,
                word,
                ha="right",
                va="center",
                fontsize=12,
                fontproperties=hindi_font
            )

            # Slash
            ax.text(
                x,
                y,
                "/",
                ha="center",
                va="center",
                fontsize=12,
                fontproperties=english_font
            )

            # POS tag
            ax.text(
                x + 0.025,
                y,
                tag,
                ha="left",
                va="center",
                fontsize=12,
                fontproperties=english_font
            )

        else:
```

```
            ax.text(
                x,
                y,
                str(tree),
                ha="center",
                va="center",
                fontsize=12,
                fontproperties=english_font
            )

# ============================================================
# CREATE GRAPH
# ============================================================

fig, ax = plt.subplots(figsize=(16, 8))

ax.axis("off")

draw_tree(tree, ax)

ax.set_title(
    "Hindi Chunking Tree",
    fontsize=16,
    fontproperties=english_font
)

plt.tight_layout()

# Save graph
plt.savefig(
    "hindi_chunking_tree.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

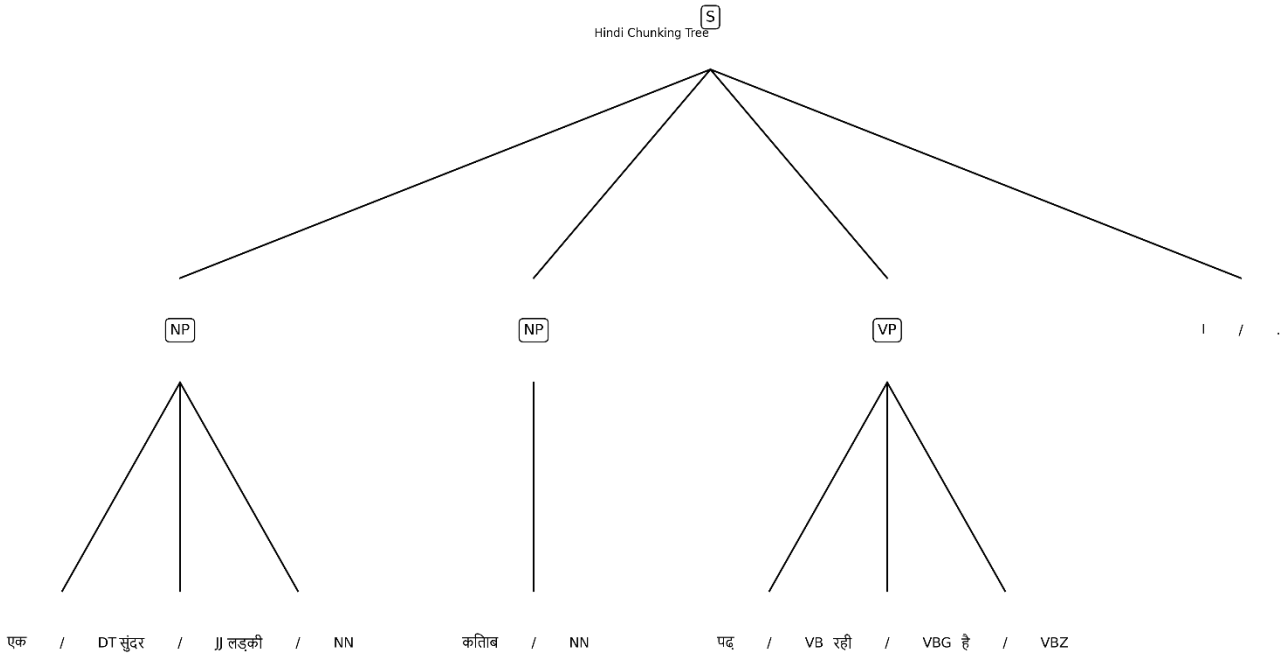
print("\nGraph saved as: hindi_chunking_tree.png")
```

Output

```
Hindi POS Tags:
एक -> DT
सुंदर -> JJ
लड़की -> NN
किताब -> NN
पढ़ -> VB
रही -> VBG
है -> VBZ
। -> .

Chunk Tree:
(S
```

(NP एक/DT सुंदर/JJ लड़की/NN)
(NP किताब/NN)
(VP पढ़/VB रही/VBG है/VBZ)
।/.)



SpaCy

```
import spacy
from spacy.matcher import Matcher
from spacy.tokens import Span
from spacy import displacy
import webbrowser

# Load multilingual spaCy model
nlp = spacy.load("xx_ent_wiki_sm")

# Hindi sentence
sentence = "एक सुंदर लड़की किताब पढ़ रही है।"

doc = nlp(sentence)

# ============================================================
# TOKENIZATION
# ============================================================

print("Hindi Tokens:")

for token in doc:
    print(token.text)

# ============================================================
# HINDI CHUNKING
# ============================================================
```

```
matcher = Matcher(nlp.vocab)

# Noun Phrase: एक सुंदर लड़की
pattern_np1 = [
    {"LOWER": "एक"},
    {"LOWER": "सुंदर"},
    {"LOWER": "लड़की"}
]

# Noun Phrase: किताब
pattern_np2 = [
    {"LOWER": "किताब"}
]

# Verb Phrase: पढ़ रही है
pattern_vp = [
    {"LOWER": "पढ़"},
    {"LOWER": "रही"},
    {"LOWER": "है"}
]

matcher.add("NP", [pattern_np1, pattern_np2])
matcher.add("VP", [pattern_vp])

matches = matcher(doc)

print("\nHindi Chunks:")

chunks = []

for match_id, start, end in matches:

    label = nlp.vocab.strings[match_id]

    chunk_text = doc[start:end].text

    print(
        f"{label} -> {chunk_text}"
    )

    chunks.append(
        Span(
            doc,
            start,
            end,
            label=label
        )
    )

# ============================================================
# CHUNK VISUALIZATION
# ============================================================
```

```
doc.spans["chunks"] = chunks

html = displacy.render(
    doc,
    style="span",
    page=True,
    options={
        "spans_key": "chunks",
        "distance": 100
    }
)

# ============================================================
# SAVE HTML
# ============================================================

output_file = "spacy_hindi_chunking.html"

with open(
    output_file,
    "w",
    encoding="utf-8"
) as f:

    f.write(html)

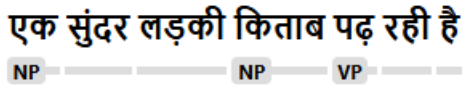
print("\nGraph saved successfully:")
print(output_file)

# Open HTML automatically
webbrowser.open(
    output_file
)
```

Output

```
Hindi Tokens:
एक
सुंदर
लड़की
किताब
पढ़
रही
है
।

Hindi Chunks:
NP -> एक सुंदर लड़की
NP -> किताब
VP -> पढ़ रही है
```



Stanza

```
import stanza
from graphviz import Digraph
import os

# Download Hindi model
stanza.download("hi")

# Hindi NLP pipeline
nlp = stanza.Pipeline(
    lang="hi",
    processors="tokenize,pos,lemma,depparse"
)

# Hindi sentence
sentence = "एक सुंदर लड़की किताब पढ़ रही है।"

doc = nlp(sentence)

# ============================================================
# POS TAGGING
# ============================================================

print("Hindi POS Tags:")

for sent in doc.sentences:
    for word in sent.words:
        print(
            word.text,
            "->",
            word.upos,
            "->",
            word.xpos
        )

# ============================================================
# DEPENDENCY RELATIONS
# ============================================================

print("\nDependency Relations:")

for sent in doc.sentences:

    for word in sent.words:

        if word.head == 0:
            head_word = "ROOT"
```

```
        else:
            head_word = sent.words[word.head - 1].text

        print(
            word.text,
            "->",
            word.deprel,
            "->",
            head_word
        )

# ============================================================
# CREATE DEPENDENCY GRAPH
# ============================================================

dot = Digraph("Hindi Dependency Tree")

dot.attr(
    rankdir="TB",
    bgcolor="white"
)

# Hindi-compatible font
dot.attr(
    "node",
    fontname="Nirmala UI",
    fontsize="14"
)

dot.attr(
    "edge",
    fontname="Nirmala UI",
    fontsize="12"
)

# Create nodes
for sent in doc.sentences:

    for word in sent.words:

        label = f"{word.text}\n{word.upos}"

        dot.node(
            str(word.id),
            label,
            shape="box"
        )

# Create dependency edges
for sent in doc.sentences:

    for word in sent.words:

        if word.head != 0:
```

```
        dot.edge(
            str(word.head),
            str(word.id),
            label=word.deprel
        )

# ============================================================
# SAVE GRAPH
# ============================================================

output_file = dot.render(
    "stanza_hindi_dependency_tree",
    format="png",
    cleanup=True
)

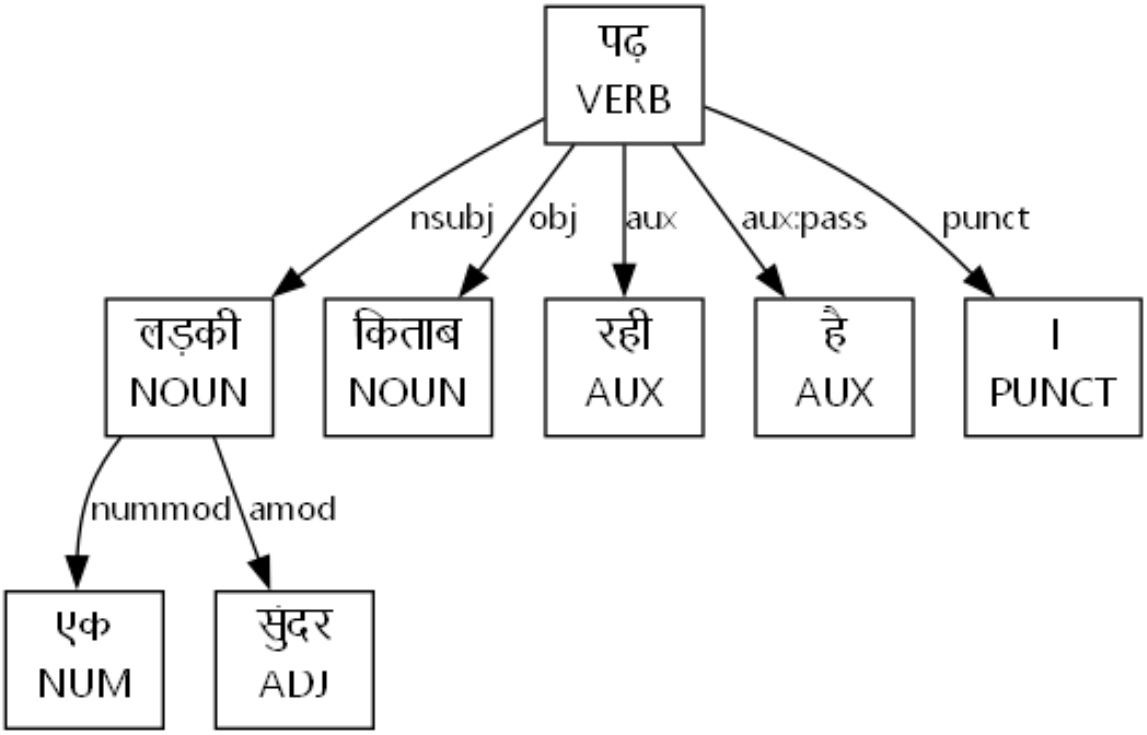
print("\nGraph saved successfully:")
print(output_file)

# Open graph automatically
os.startfile(output_file)
```

Output

```
Hindi POS Tags:
एक -> NUM -> QC
सुंदर -> ADJ -> JJ
लड़की -> NOUN -> NN
किताब -> NOUN -> NN
पढ़ -> VERB -> VM
रही -> AUX -> VAUX
है -> AUX -> VAUX
। -> PUNCT -> SYM

Dependency Relations:
एक -> nummod -> लड़की
सुंदर -> amod -> लड़की
लड़की -> nsubj -> पढ़
किताब -> obj -> पढ़
पढ़ -> root -> ROOT
रही -> aux -> पढ़
है -> aux:pass -> पढ़
। -> punct -> पढ़
```



Marathi
Nltk

```
import nltk
import matplotlib.pyplot as plt
from matplotlib.font_manager import FontProperties

# Marathi sentence
sentence = "एक सुंदर मुलगी पुस्तक वाचत आहे."

# Marathi words with manual POS tags
tagged_words = [
    ("एक", "DT"),
    ("सुंदर", "JJ"),
    ("मुलगी", "NN"),
    ("पुस्तक", "NN"),
    ("वाचत", "VB"),
    ("आहे", "VBZ"),
    ("।", ".")
]

print("Marathi POS Tags:")
print(tagged_words)

# Chunking grammar
grammar = r"""
    NP: {<DT><JJ><NN>}
    NP: {<NN>}
    VP: {<VB><VBZ>}
"""
```

```
# Create chunk parser
chunk_parser = nltk.RegexpParser(grammar)

# Create chunk tree
tree = chunk_parser.parse(tagged_words)

print("\nChunk Tree:")
print(tree)

# ------------------------------------------------
# Create graph using Matplotlib
# ------------------------------------------------

# Marathi font
font_path = r"C:\Windows\Fonts\Nirmala.ttc"
marathi_font = FontProperties(fname=font_path, size=14)
normal_font = FontProperties(size=12)

# Get tree positions
positions = {}
labels = {}

def draw_tree(node, x, y, width):
    if isinstance(node, tuple):
        word, tag = node
        positions[id(node)] = (x, y)
        labels[id(node)] = (word, tag)
        return

    children = node
    label = node.label()

    if len(children) == 0:
        positions[id(node)] = (x, y)
        labels[id(node)] = (label, "")
        return

    child_width = width / len(children)
    child_positions = []

    for i, child in enumerate(children):
        child_x = x - width / 2 + child_width / 2 + i * child_width
        draw_tree(child, child_x, y - 1.5, child_width)
        child_positions.append(
            positions.get(id(child))
        )

    positions[id(node)] = (x, y)
    labels[id(node)] = (label, "")

    for child_pos in child_positions:
        if child_pos:
            plt.plot(
                [x, child_pos[0]],
```

```
            [y - 0.1, child_pos[1] + 0.2],
            linewidth=1
        )

plt.figure(figsize=(14, 7))

draw_tree(tree, 7, 5, 14)

# Draw labels
for key, (x, y) in positions.items():

    if key not in labels:
        continue

    label = labels[key]

    if isinstance(label, tuple):
        word, tag = label

        # Word in Marathi font
        plt.text(
            x,
            y,
            word,
            ha="center",
            va="center",
            fontproperties=marathi_font,
            bbox=dict(
                boxstyle="round,pad=0.4",
                facecolor="white",
                edgecolor="black"
            )
        )

        # POS tag
        plt.text(
            x,
            y - 0.45,
            tag,
            ha="center",
            va="center",
            fontproperties=normal_font
        )

    else:
        plt.text(
            x,
            y,
            label,
            ha="center",
            va="center",
            fontproperties=normal_font,
            fontsize=13,
            fontweight="bold"
        )
```

```
plt.title(
    "Marathi NLTK Chunking",
    fontproperties=normal_font,
    fontsize=16
)

plt.axis("off")
plt.tight_layout()

# Save graph
plt.savefig(
    "nltk_marathi_chunking.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

print("\nGraph saved successfully:")
print("nltk_marathi_chunking.png")
```

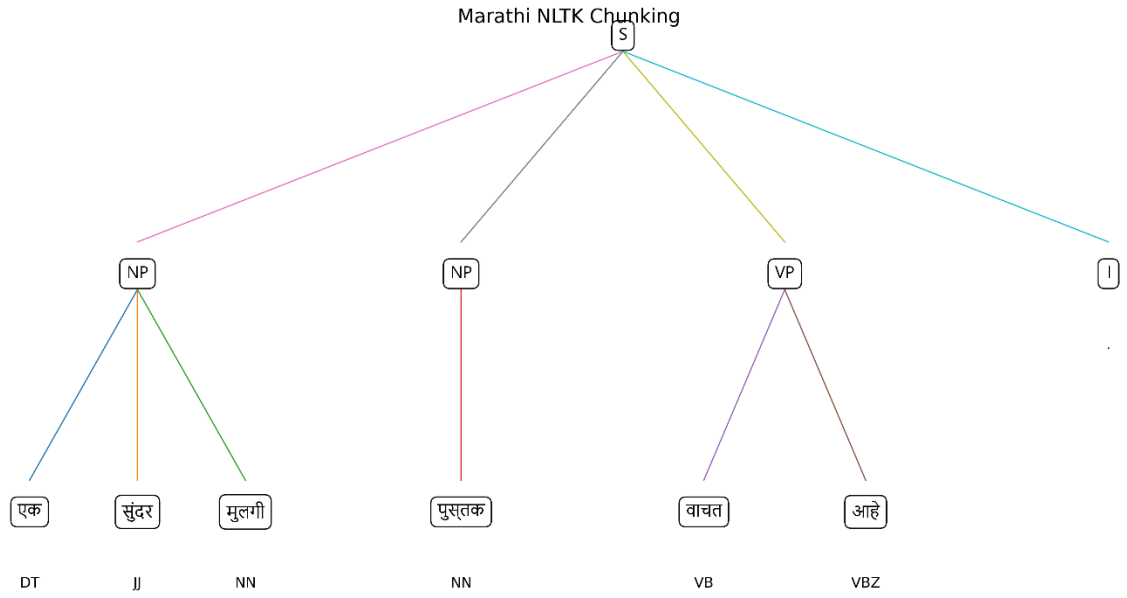
Output

Marathi POS Tags:

[('एक', 'DT'), ('सुंदर', 'JJ'), ('मुलगी', 'NN'), ('पुस्तक', 'NN'), ('वाचत', 'VB'), ('आहे', 'VBZ'), ('।', '.')]

Chunk Tree:

```
(S
  (NP एक/DT सुंदर/JJ मुलगी/NN)
  (NP पुस्तक/NN)
  (VP वाचत/VB आहे/VBZ)
  ।/.)
```



SpaCy

```
import spacy
from spacy.matcher import Matcher
from spacy.tokens import Span
from spacy import displacy
import webbrowser

# Multilingual spaCy model
nlp = spacy.load("xx_ent_wiki_sm")

# Marathi sentence
sentence = "एक सुंदर मुलगी पुस्तक वाचत आहे."

doc = nlp(sentence)

# ------------------------------
# Tokens
# ------------------------------

print("Marathi Tokens:")

for token in doc:
    print(token.text)

# ------------------------------
# Matcher
# ------------------------------

matcher = Matcher(nlp.vocab)

# Noun Phrase: एक सुंदर मुलगी
pattern_np1 = [
    {"LOWER": "एक"},
    {"LOWER": "सुंदर"},
    {"LOWER": "मुलगी"}
]

# Noun Phrase: पुस्तक
pattern_np2 = [
    {"LOWER": "पुस्तक"}
]

# Verb Phrase: वाचत आहे
pattern_vp = [
    {"LOWER": "वाचत"},
    {"LOWER": "आहे"}
]

matcher.add("NP", [pattern_np1, pattern_np2])
matcher.add("VP", [pattern_vp])

matches = matcher(doc)
```

```
print("\nMarathi Chunks:")

chunks = []

for match_id, start, end in matches:

    label = nlp.vocab.strings[match_id]

    chunk_text = doc[start:end].text

    print(f"{label} -> {chunk_text}")

    chunks.append(
        Span(
            doc,
            start,
            end,
            label=label
        )
    )

# ----------------------------
# Create visualization
# ----------------------------

doc.spans["chunks"] = chunks

html = displacy.render(
    doc,
    style="span",
    page=True,
    options={
        "spans_key": "chunks",
        "distance": 100
    }
)

# Save HTML
output_file = "spacy_marathi_chunking.html"

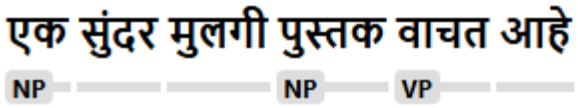
with open(
    output_file,
    "w",
    encoding="utf-8"
) as f:
    f.write(html)

print("\nGraph saved successfully:")
print(output_file)

# Open in browser
webbrowser.open(output_file)
```

Output
Marathi Tokens:
एक
सुंदर
मुलगी
पुस्तक
वाचत
आहे
.

Marathi Chunks:
NP -> एक सुंदर मुलगी
NP -> पुस्तक
VP -> वाचत आहे



Stanza

```
import stanza
from graphviz import Digraph
import os

# Graphviz path
os.environ["PATH"] += os.pathsep + r"C:\Program Files\Graphviz\bin"

# Download Marathi model
stanza.download("mr")

# Create Marathi NLP pipeline
nlp = stanza.Pipeline(
    lang="mr",
    processors="tokenize,pos,lemma,depparse"
)

# Marathi sentence
sentence = "एक सुंदर मुलगी पुस्तक वाचत आहे."

doc = nlp(sentence)

# ------------------------------
# POS Tags
# ------------------------------

print("Marathi POS Tags:")

for sent in doc.sentences:
    for word in sent.words:
        print(
            word.text,
```

```
            "->",
            word.upos,
            "->",
            word.xpos
        )

# ------------------------------
# Dependency Relations
# ------------------------------

print("\nDependency Relations:")

for sent in doc.sentences:
    for word in sent.words:

        if word.head == 0:
            head_word = "ROOT"
        else:
            head_word = sent.words[word.head - 1].text

        print(
            word.text,
            "->",
            word.deprel,
            "->",
            head_word
        )

# ------------------------------
# Create Graphviz graph
# ------------------------------

dot = Digraph("Marathi Dependency Tree")

dot.attr(
    rankdir="TB",
    bgcolor="white"
)

# Marathi-compatible font
dot.attr(
    "node",
    fontname="Nirmala UI",
    fontsize="14"
)

dot.attr(
    "edge",
    fontname="Nirmala UI",
    fontsize="12"
)

# Create nodes
for sent in doc.sentences:

    for word in sent.words:

        label = f"{word.text}\n{word.upos}"

        dot.node(
```

```
            str(word.id),
            label,
            shape="box"
        )

# Create edges
for sent in doc.sentences:

    for word in sent.words:

        if word.head != 0:

            dot.edge(
                str(word.head),
                str(word.id),
                label=word.deprel
            )

# ----------------------------
# Save graph
# ----------------------------

output_file = dot.render(
    "stanza_marathi_dependency_tree",
    format="png",
    cleanup=True
)

print("\nGraph saved successfully:")
print(output_file)

# Open PNG
os.startfile(output_file)

#python stanza_marathi.py
```

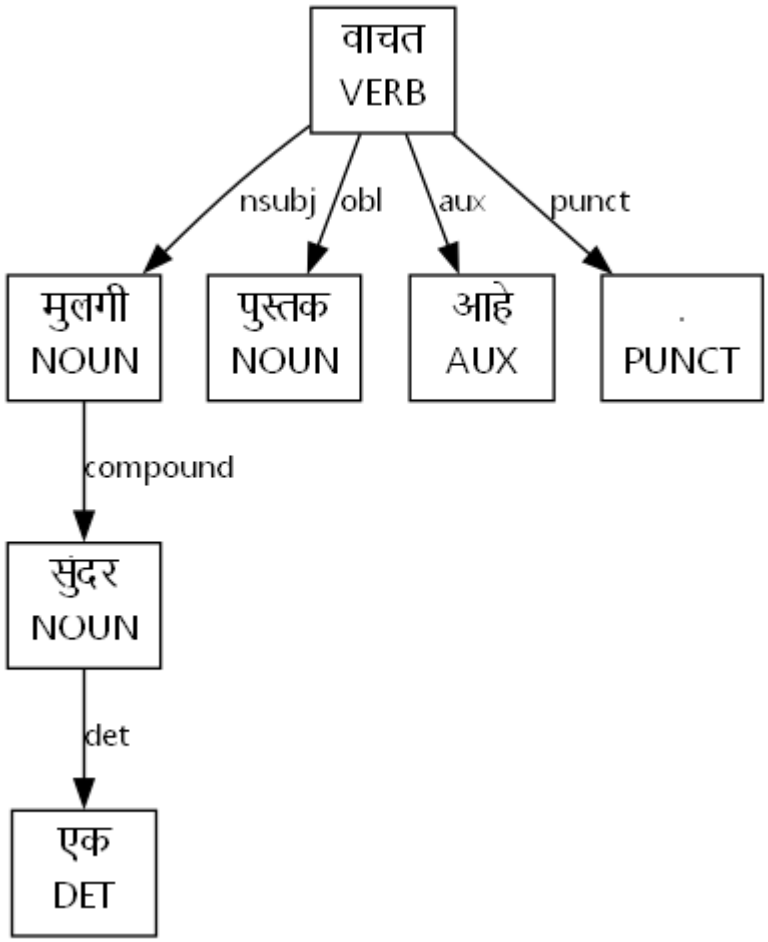
Output

Marathi POS Tags:

एक -> DET -> None
सुंदर -> NOUN -> None
मुलगी -> NOUN -> None
पुस्तक -> NOUN -> None
वाचत -> VERB -> None
आहे -> AUX -> None
. -> PUNCT -> None

Dependency Relations:

एक -> det -> सुंदर
सुंदर -> compound -> मुलगी
मुलगी -> nsubj -> वाचत
पुस्तक -> obl -> वाचत
वाचत -> root -> ROOT
आहे -> aux -> वाचत
. -> punct -> वाचत


वाचत
VERB
nsubj
obl
aux
punct
मुलगी
NOUN
पुस्तक
NOUN
आहे
AUX
.
PUNCT
compound
सुंदर
NOUN
det
एक
DET